

Roteiro — Adaptar o Trabalho Final de Python para SQL e Elasticsearch

Disciplina: Banco de Dados para Cientistas de Dados

Base: o seu **próprio trabalho final de Python/ML**, que hoje lê os dados de **CSV/Excel**.

1. Objetivo

Você **não vai refazer** o trabalho final de Python. Vai **adaptá-lo**: hoje ele lê os dados de um arquivo CSV/Excel; o objetivo agora é fazer ele ler de um **banco de dados**.

A entrega são **duas cópias do seu app Streamlit**:

- `app_sql.py` — consome os dados de um **banco SQL** (relacional, normalizado).
- `app_elastic.py` — consome os dados de um **Elasticsearch** (NoSQL, com template/mapping adequado).

A análise, os gráficos, as features e a pergunta de negócio **continuam exatamente os mesmos**. A única coisa que muda entre as duas cópias é **de onde os dados vêm**.

No caminho, você cria dois artefatos novos: a **modelagem SQL** (seguindo as regras de normalização da Aula 1) e o **template do Elastic** (seguindo o mapping da Aula 2), ambos para **os seus próprios dados**.

2. O que muda e o que NÃO muda

Continua igual (do trabalho original)	Muda nesta adaptação
Pergunta de negócio / problema	Fonte de dados: CSV/Excel → SQL e → Elasticsearch
Limpeza com Pandas/Numpy	Onde a limpeza roda (passa a rodar uma vez , na carga)
Features criadas	—
Gráficos Matplotlib e layout Streamlit	—

Continua igual (do trabalho original)	Muda nesta adaptação
Relatório (estrutura)	Ganha 2 seções novas: modelagem SQL e template Elastic

Mensagem central: o código de **gráficos é copiado igual** nas duas versões. Só trocam as **linhas que carregam o DataFrame**.

3. Passo 0 — Ache o "ponto de leitura" no seu código atual

Abra o seu `app.py` atual e localize a linha onde os dados entram. Vai ser algo como:

```
df = pd.read_csv("dados.csv") # ou pd.read_excel("dados.xlsx")
```

Essa é a **única coisa que você vai substituir**. Identifique também quais DataFrames cada gráfico usa. Tudo o que vem depois (limpeza, features, `plt`, `st.pyplot`) permanece.

4. Passo 1 — Modele e carregue seus dados no SQL (normalização)

Aula 1: ER, chaves (PK/FK/composta/candidata), normalização 1FN→3FN, DDL, índices.

4.1 Como normalizar o seu CSV

Seu arquivo é uma **tabela "achatada"** (tudo numa linha só). Aplique as três formas normais aos **seus** dados:

1. **1FN** — sem grupos repetidos; cada célula com um valor atômico. Se houver listas/colunas repetidas (`item1`, `item2`, `item3`), vire **linhas** numa tabela separada.
2. **2FN** — remova dependências parciais. Se você tiver chave composta, atributos que dependem de só **parte** da chave vão para tabela própria.
3. **3FN** — remova dependências transitivas. Atributo que depende de **outro atributo não-chave** (ex.: nome da categoria depende do código da categoria, não do produto) vira **dimensão própria**.

Padrão prático que cobre quase todo trabalho: identifique a **entidade-evento (fato)** do seu dado (uma venda, uma medição, um log, um jogo, uma transação) e separe em **dimensões** tudo que se repete e tem atributos próprios (cliente, produto, local, categoria...).

4.2 DDL genérico — adapte para `schema.sql`

```
-- EXEMPLO GENÉRICO — troque nomes/tipos pelos do SEU dataset

-- DIMENSÃO: algo que se repete e tem atributos próprios
-- (cliente, produto, sensor, categoria, time, país...)
CREATE TABLE dimensao_x (
    x_id    INTEGER PRIMARY KEY,           -- chave primária (PK)
    nome    TEXT NOT NULL,
    -- ... atributos que dependem SÓ de x_id (3FN) ...
    UNIQUE (nome)                         -- chave candidata, se
existir
);

-- FATO: cada linha = um evento; aponta para as dimensões via FK
CREATE TABLE fato (
    fato_id INTEGER PRIMARY KEY,
    x_id    INTEGER NOT NULL REFERENCES dimensao_x(x_id), -- chave
estrangeira (FK)
    data    TIMESTAMP,
    valor    NUMERIC(12,2) CHECK (valor >= 0), -- constraint = dado limpo
    -- ... demais métricas do evento ...
);

-- Relação N:M (ex.: pedido tem vários produtos) -> tabela ASSOCIATIVA com
PK COMPOSTA
-- CREATE TABLE fato_item (
--     fato_id    INTEGER NOT NULL REFERENCES fato(fato_id),
--     item_seq   INTEGER NOT NULL,
--     produto_id INTEGER NOT NULL REFERENCES dimensao_x(x_id),
--     PRIMARY KEY (fato_id, item_seq)
-- );

-- ÍNDICES nas FKs e em colunas muito filtradas/ordenadas (B-tree -> O(log
n))
CREATE INDEX idx_fato_x    ON fato(x_id);
CREATE INDEX idx_fato_data ON fato(data);
```

4.3 Carga única — `carregar_sql.py`

```
import sqlite3
# df_dim, df_fato = DataFrames já limpos (resultado da sua limpeza
Pandas/Numpy)
con = sqlite3.connect("dados.db")
```

```

con.executescript(open("schema.sql", encoding="utf-8").read())
df_dim.to_sql("dimensao_x", con, if_exists="append", index=False)    #
dimensões 1º
df_fato.to_sql("fato", con, if_exists="append", index=False)    # fato
depois (FK)
con.commit(); con.close()

```

Carregue **dimensões antes do fato** (senão a FK falha). Em SQLite, `PRAGMA foreign_keys = ON;` para validar.

5. Passo 2 — Modele e indexe seus dados no Elasticsearch (template)

Aula 2: documento JSON desnormalizado, `text × keyword`, `date`, `geo_point`, `nested`, `helpers.bulk()`.

5.1 A lógica é o oposto do SQL: desnormalizar

No Elastic **não há JOIN**. Em vez de dividir em tabelas, você junta tudo num **único índice** onde cada documento já vem completo. A análise vira **agregação sobre documentos**.

5.2 Decisão que mais importa: `text × keyword`

- `keyword` → campos usados em **filtro**, **agregação** (`terms`) e **ordenação** (categorias, UF, status, ids).
- `text` → campos de **busca full-text** com relevância BM25 (descrições, comentários).
- Se precisar dos dois, use `text` com subcampo `.raw` do tipo `keyword`.

5.3 Template genérico — adapte para `meu-indice-template.json`

```

{
  "index_patterns": ["meu-indice*"],
  "template": {
    "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
    "mappings": {
      "properties": {
        "id": { "type": "keyword" },
        "categoria": { "type": "keyword" },
        "descricao": { "type": "text" },
        "data": { "type": "date" },
        "valor": { "type": "double" },
        "ativo": { "type": "boolean" },

```

```

    "local": { "type": "geo_point" },
    "itens": {
        "type": "nested",
        "properties": {
            "produto": { "type": "keyword" },
            "preco": { "type": "double" }
        }
    }
}
}
}
}
}
}
}

```

5.4 Ingestão única — `indexar_elastic.py`

```

import json
from elasticsearch import Elasticsearch, helpers

es = Elasticsearch("http://localhost:9200")

tpl = json.load(open("meu-indice-template.json", encoding="utf-8"))
es.indices.put_index_template(name="meu-indice-template", **tpl)
es.indices.create(index="meu-indice") # herda o mapping do
template

def acoes(df): # df = DataFrame já limpo
    (1 documento por linha)
    for doc in df.to_dict(orient="records"):
        yield {"_index": "meu-indice", "_source": doc}

helpers.bulk(es, acoes(df_docs)) # df_docs = seus dados
limpos; ingestão em lote

```

6. Passo 3 — Cópia 1: `app_sql.py`

Copie o seu `app.py` para `app_sql.py` e troque **só a leitura**.

Antes (original, CSV):

```

import pandas as pd
df = pd.read_csv("dados.csv")
# ... gráficos ...

```

Depois (mínimo — carrega a tabela e mantém sua lógica pandas):

```
from sqlalchemy import create_engine
import pandas as pd
engine = create_engine("sqlite:///dados.db")
df = pd.read_sql("SELECT * FROM fato", engine)
# ... MESMOS gráficos, sem mudar nada ...
```

Depois (recomendado — empurra a agregação para o SQL; exercita JOIN/GROUP BY da Aula 1):

```
df_resumo = pd.read_sql("""
    SELECT d.nome AS categoria, SUM(f.valor) AS total
    FROM fato f
    JOIN dimensao_x d ON d.x_id = f.x_id
    GROUP BY d.nome
    ORDER BY total DESC
""", engine)
```

Segurança (Aula 1): se algum filtro vier de um `st.selectbox / st.text_input`, **não concatene** na string SQL. Use parâmetros: `pd.read_sql(text("... WHERE estado = :uf"), engine, params={"uf": uf})`.

7. Passo 4 — Cópia 2: `app_elastic.py`

Copie o seu `app.py` para `app_elastic.py` e troque **só a leitura** — devolvendo o **mesmo DataFrame** do `app_sql.py`.

Depois (mínimo — carrega os documentos):

```
from elasticsearch import Elasticsearch
import pandas as pd
es = Elasticsearch("http://localhost:9200")
hits = es.search(index="meu-indice", size=10000, query={"match_all": {}})
["hits"]["hits"]
df = pd.json_normalize([h["_source"] for h in hits])
# ... MESMOS gráficos ...
```

Atenção: por padrão o Elastic devolve no máximo **10.000 documentos** por busca. Para bases maiores, **não puxe tudo** — use agregação (abaixo) ou `search_after / scroll`.

Depois (recomendado — agregação no servidor; gera o MESMO df_resumo do SQL):

```
resp = es.search(
    index="meu-indice",
    size=0,
    aggs={
        "por_categoria": {
            "terms": {"field": "categoria", "size": 20},
            "aggs": {"total": {"sum": {"field": "valor"}}}
        }
    }
)
b = resp["aggregations"]["por_categoria"]["buckets"]
df_resumo = pd.DataFrame([{"categoria": x["key"], "total": x["total"]
    ["value"]} for x in b])
```

Equivalência das operações (mesma análise, dois mundos)

Sua análise	SQL (app_sql.py)	Elastic (app_elastic.py)
Total por grupo	GROUP BY ... SUM/AVG	terms + sum / avg
Série temporal	GROUP BY strftime(...)	date_histogram
Contagem por categoria	GROUP BY ... COUNT	terms
Filtro por intervalo	WHERE x BETWEEN a AND b	range (gte/lte)
Busca em texto livre	LIKE '%...%' (lento)	match (BM25, com relevância)
Mapa de calor (se tiver geo)	aproximado por região	geohash_grid (nativo)

8. Erros comuns

- **FK falhando na carga SQL:** carregue as dimensões antes do fato.
- **Elastic só traz 10.000 linhas de uma vez:** prefira **agregação no servidor** a puxar todos os documentos ou use search scroll.
- **Agregação terms em campo text dá erro:** use keyword (ou o subcampo .raw) para agregar/filtrar.
- **Gráficos diferentes entre as duas versões:** o DataFrame de cada análise tem que ter as **mesmas colunas e o mesmo formato** nas duas cópias — ajuste a query, não o gráfico.

- **Filtro do usuário direto na string SQL:** sempre parâmetros nomeados (risco de SQL injection — Aula 1).